JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Enum DayOfWeek

java.lang.Object
    java.lang.Enum<DayOfWeek>
        java.time.DayOfWeek

All Implemented Interfaces:

Serializable, Comparable<DayOfWeek>, TemporalAccessor, TemporalAdjuster

---

```
public enum DayOfWeek
extends Enum<DayOfWeek>
implements TemporalAccessor, TemporalAdjuster
```

A day-of-week, such as 'Tuesday'.

`DayOfWeek` is an enum representing the 7 days of the week - Monday, Tuesday, Wednesday, Thursday, Friday, Saturday and Sunday.

In addition to the textual enum name, each day-of-week has an `int` value. The `int` value follows the ISO-8601 standard, from 1 (Monday) to 7 (Sunday). It is recommended that applications use the enum rather than the `int` value to ensure code clarity.

This enum provides access to the localized textual form of the day-of-week. Some locales also assign different numeric values to the days, declaring Sunday to have the value 1, however this class provides no support for this. See `WeekFields` for localized week-numbering.

Do not use `ordinal()` to obtain the numeric representation of `DayOfWeek`. Use `getValue()` instead.

This enum represents a common concept that is found in many calendar systems. As such, this enum may be used by any calendar system that has the day-of-week concept defined exactly equivalent to the ISO calendar system.

Implementation Requirements:

`This is an immutable and thread-safe enum.`

Since:

1.8

Enum Constant Summary

Enum Constants

Enum Constant	Description
FRIDAY	The singleton instance for the day-of-week of Friday.
MONDAY	The singleton instance for the day-of-week of Monday.
SATURDAY	The singleton instance for the day-of-week of Saturday.
SUNDAY	The singleton instance for the day-of-week of Sunday.
THURSDAY	The singleton instance for the day-of-week of Thursday.
TUESDAY	The singleton instance for the day-of-week of Tuesday.
WEDNESDAY	The singleton instance for the day-of-week of Wednesday.

Method Summary

Concrete Methods **Instance Method**

Modifier and Type	Method	Description
Temporal	**adjustInto**(**Temporal** temporal)	Adjusts the specified temporal object to have this day-of-week.
static **DayOfWeek**	**from**(**TemporalAccessor** temporal)	Obtains an instance of DayOfWeek from a temporal object.
int	**get**(**TemporalField** field)	Gets the value of the specified field from this

		day-of-week as an `int`.
`String`	`getDisplayName(TextStyle style, Locale locale)`	Gets the textual representation, such as 'Mon' or 'Friday'.
`long`	`getLong(TemporalField field)`	Gets the value of the specified field from this day-of-week as a `long`.
`int`	`getValue()`	Gets the day-of-week `int` value.
`boolean`	`isSupported(TemporalField field)`	Checks if the specified field is supported.
`DayOfWeek`	`minus(long days)`	Returns the day-of-week that is the specified number of days before this one.
`static DayOfWeek`	`of(int dayOfWeek)`	Obtains an instance of `DayOfWeek` from an `int` value.
`DayOfWeek`	`plus(long days)`	Returns the day-of-week that is the specified number of days after this one.

<R> R	**query**(**TemporalQuery**<R> query)	Queries this day-of-week using the specified query.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
static **DayOfWeek**	**valueOf**(**String** name)	Returns the enum constant of this type with the specified name.
static **DayOfWeek**[]	**values**()	Returns an array containing the constants of this enum type, in the order they are declared.

Methods inherited from class java.lang.Enum

clone, compareTo, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf

Methods inherited from class java.lang.Object

getClass, notify, notifyAll, wait, wait, wait

Enum Constant Detail

MONDAY

```
public static final DayOfWeek MONDAY
```

The singleton instance for the day-of-week of Monday. This has the numeric value of 1.

TUESDAY

```
public static final DayOfWeek TUESDAY
```

The singleton instance for the day-of-week of Tuesday. This has the numeric value of 2.

WEDNESDAY

```
public static final DayOfWeek WEDNESDAY
```

The singleton instance for the day-of-week of Wednesday. This has the numeric value of 3.

THURSDAY

```
public static final DayOfWeek THURSDAY
```

The singleton instance for the day-of-week of Thursday. This has the numeric value of 4.

FRIDAY

```
public static final DayOfWeek FRIDAY
```

The singleton instance for the day-of-week of Friday. This has the numeric value of 5.

SATURDAY

```
public static final DayOfWeek SATURDAY
```

The singleton instance for the day-of-week of Saturday. This has the numeric value of 6.

SUNDAY

```
public static final DayOfWeek SUNDAY
```

The singleton instance for the day-of-week of Sunday. This has the numeric value of 7.

Method Detail

values

```
public static DayOfWeek[] values()
```

Returns an array containing the constants of this enum type, in the order they are declared. This method may be used to iterate over the constants as follows:

```
for (DayOfWeek c : DayOfWeek.values())
    System.out.println(c);
```

Returns:

an array containing the constants of this enum type, in the order they are declared

valueOf

```
public static DayOfWeek valueOf(String name)
```

Returns the enum constant of this type with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this type. (Extraneous whitespace characters are not permitted.)

Parameters:

name - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

IllegalArgumentException - if this enum type has no constant with the specified name

NullPointerException - if the argument is null

of

```
public static DayOfWeek of(int dayOfWeek)
```

Obtains an instance of DayOfWeek from an int value.

DayOfWeek is an enum representing the 7 days of the week. This factory allows the enum to be obtained from the int value. The int value follows the ISO-8601 standard, from 1 (Monday) to 7 (Sunday).

Parameters:

dayOfWeek - the day-of-week to represent, from 1 (Monday) to 7 (Sunday)

Returns:

the day-of-week singleton, not null

Throws:

DateTimeException - if the day-of-week is invalid

from

public static DayOfWeek from(TemporalAccessor temporal)

Obtains an instance of DayOfWeek from a temporal object.

This obtains a day-of-week based on the specified temporal. A TemporalAccessor represents an arbitrary set of date and time information, which this factory converts to an instance of DayOfWeek.

The conversion extracts the DAY_OF_WEEK field.

This method matches the signature of the functional interface TemporalQuery allowing it to be used as a query via method reference, DayOfWeek::from.

Parameters:

temporal - the temporal object to convert, not null

Returns:

the day-of-week, not null

Throws:

DateTimeException - if unable to convert to a DayOfWeek

getValue

public int getValue()

Gets the day-of-week int value.

The values are numbered following the ISO-8601 standard, from 1 (Monday) to 7 (Sunday). See WeekFields.dayOfWeek() for localized week-numbering.

Returns:

the day-of-week, from 1 (Monday) to 7 (Sunday)

getDisplayName

```
public String getDisplayName(TextStyle style,
                             Locale locale)
```

Gets the textual representation, such as 'Mon' or 'Friday'.

This returns the textual name used to identify the day-of-week, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

If no textual mapping is found then the `numeric value` is returned.

Parameters:

`style - the length of the text required, not null`

`locale - the locale to use, not null`

Returns:

`the text value of the day-of-week, not null`

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this day-of-week can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

If the field is `DAY_OF_WEEK` then this method returns true. All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field is supported on this day-of-week, false if not`

range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This day-of-week is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is `DAY_OF_WEEK` then the range of the day-of-week, from 1 to 7, will be returned. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field - the field to query the range for, not null`

Returns:

`the range of valid values for the field, not null`

Throws:

`DateTimeException - if the range for the field cannot be obtained`

`UnsupportedTemporalTypeException - if the field is not supported`

get

`public int get(TemporalField field)`

Gets the value of the specified field from this day-of-week as an `int`.

This queries this day-of-week for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is `DAY_OF_WEEK` then the value of the day-of-week, from 1 to 7, will be returned. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the

argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field, within the valid range of values

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an int

`ArithmeticException` - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this day-of-week as a `long`.

This queries this day-of-week for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is `DAY_OF_WEEK` then the value of the day-of-week, from 1 to 7, will be returned. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

ArithmeticException - if numeric overflow occurs

plus

public DayOfWeek plus(long days)

Returns the day-of-week that is the specified number of days after this one.

The calculation rolls around the end of the week from Sunday to Monday. The specified period may be negative.

This instance is immutable and unaffected by this method call.

Parameters:

days - the days to add, positive or negative

Returns:

the resulting day-of-week, not null

minus

public DayOfWeek minus(long days)

Returns the day-of-week that is the specified number of days before this one.

The calculation rolls around the start of the year from Monday to Sunday. The specified period may be negative.

This instance is immutable and unaffected by this method call.

Parameters:

days - the days to subtract, positive or negative

Returns:

the resulting day-of-week, not null

query

public <R> R query(TemporalQuery<R> query)

Queries this day-of-week using the specified query.

This queries this day-of-week using the specified query strategy object. The TemporalQuery object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this day-of-week.

This returns a temporal object of the same observable type as the input with the day-of-week changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` passing `ChronoField.DAY_OF_WEEK` as the field. Note that this adjusts forwards or backwards within a Monday to Sunday week. See `WeekFields.dayOfWeek()` for localized week start days. See `TemporalAdjuster` for other adjusters with more control, such as `next(MONDAY)`.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisDayOfWeek.adjustInto(temporal);
temporal = temporal.with(thisDayOfWeek);
```

For example, given a date that is a Wednesday, the following are output:

```
dateOnWed.with(MONDAY);     // two days earlier
dateOnWed.with(TUESDAY);    // one day earlier
dateOnWed.with(WEDNESDAY);  // same date
dateOnWed.with(THURSDAY);   // one day later
```

```
   dateOnWed.with(FRIDAY);     // two days later
   dateOnWed.with(SATURDAY);   // three days later
   dateOnWed.with(SUNDAY);     // four days later
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.